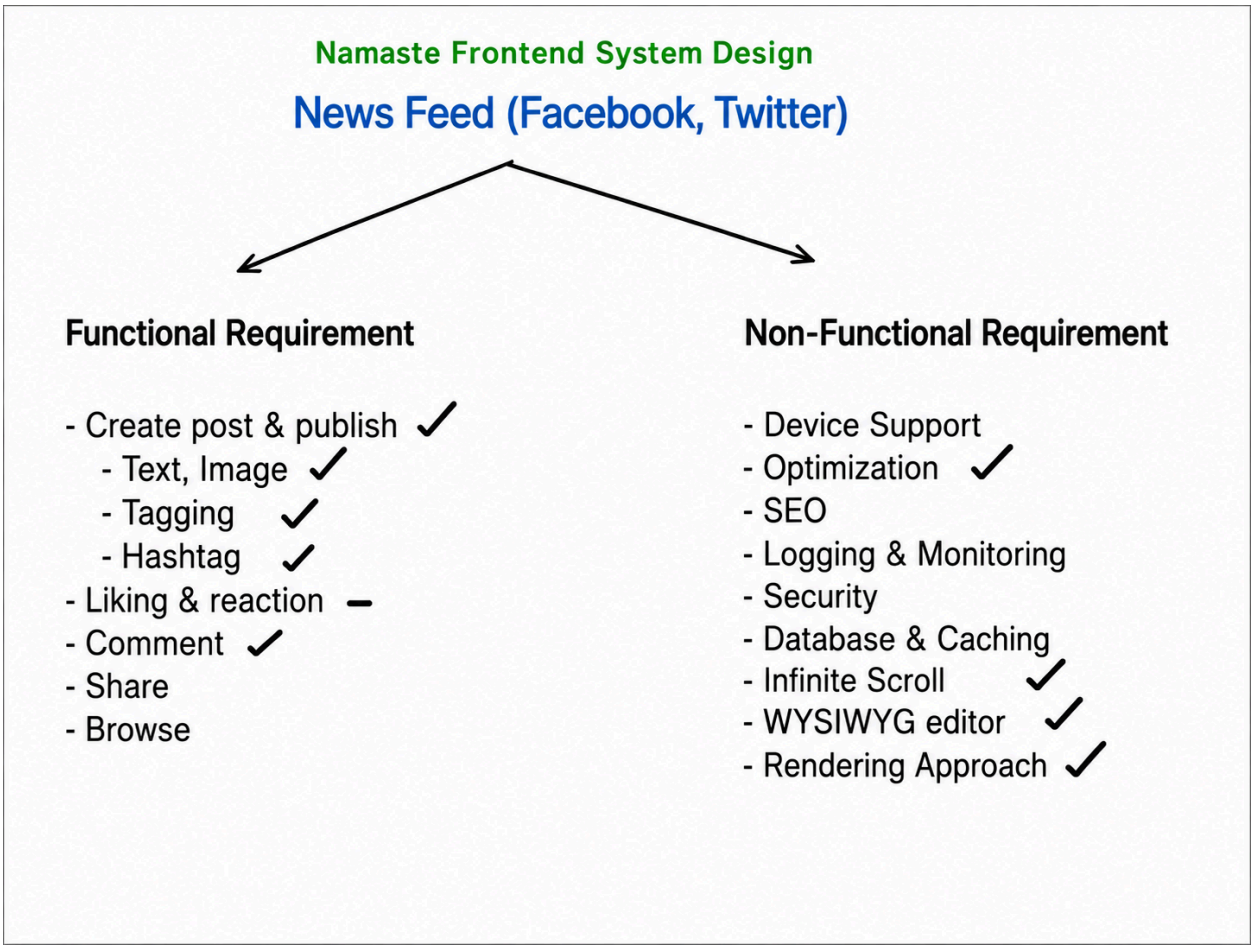
HLD - News Media Feed(Facebook)



Functional Requirements

Functional requirements define **what features the application should provide to users**.

Create Post & Publish

Users should be able to create and publish content on the platform. The system should support writing content and making it available in the feed after

submission.

Content Types

- Text posts
- Image posts

Additional Features

- User tagging (@username)
- Hashtags (#frontend)

Engagement Features

Users should be able to interact with content.

- Like or react to posts
- Comment on posts
- Share posts with others

Feed Consumption

Users should be able to browse the news feed and consume content published by other users.

Non-Functional Requirements

Non-functional requirements define **how the application should behave and perform**.

Device Support

The application should work properly across:

- Mobile devices
- Tablets
- Desktop systems

The UI should remain responsive and consistent on different screen sizes.

Optimization

Since the feed may contain a large number of posts, the frontend should be optimized for:

- Fast loading
- Smooth scrolling
- Efficient rendering

SEO

Public content should be discoverable by search engines when required.

This influences decisions related to rendering strategies and metadata handling.

Logging & Monitoring

The system should track errors and application health to help developers identify issues quickly.

Examples include API failures, frontend crashes, and performance problems.

Security

The application should protect user data and prevent common frontend security vulnerabilities.

Database & Caching

The frontend should effectively utilize caching mechanisms to reduce unnecessary network requests and improve performance.

Infinite Scroll

Instead of traditional pagination, posts should load automatically as the user scrolls through the feed.

This provides a seamless browsing experience.

WYSIWYG Editor

The post creation interface should provide a rich editing experience where users can format content and see the result while editing.

Rendering Approach

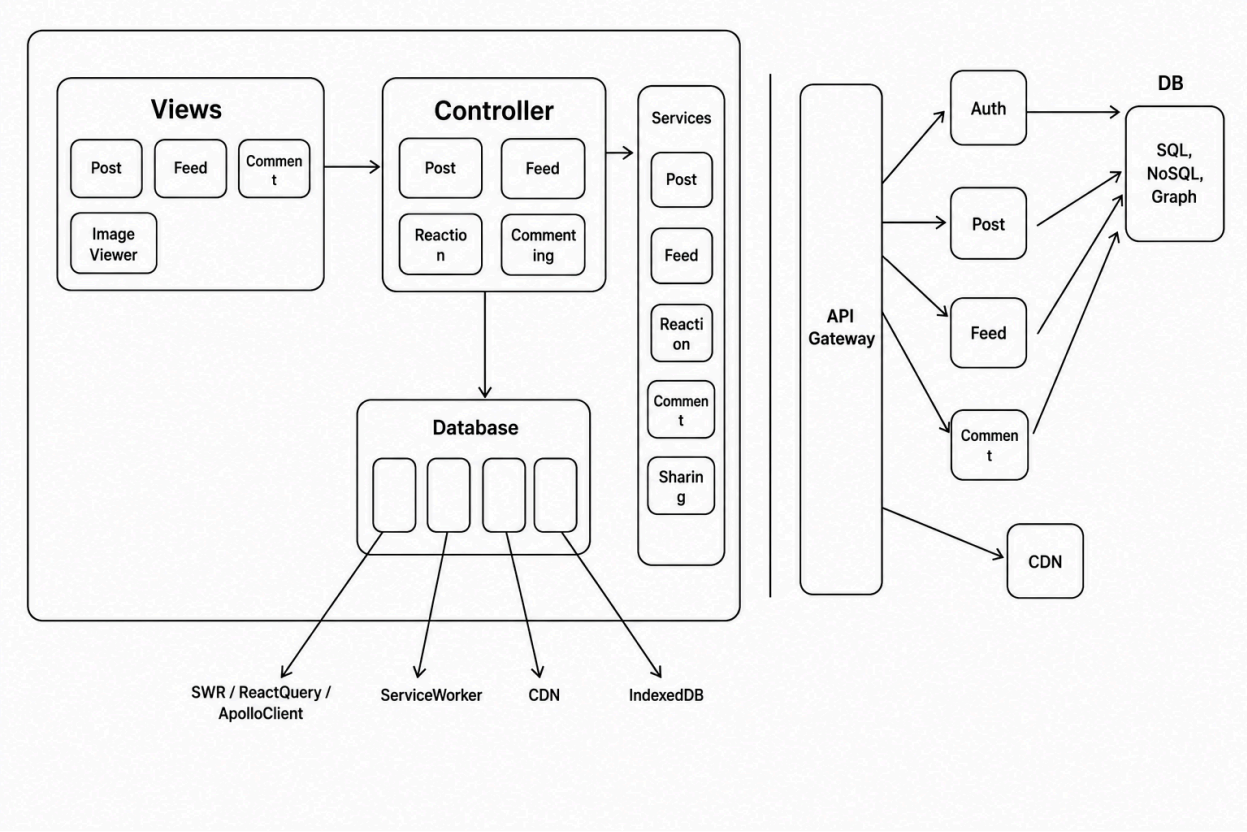
The frontend team must decide how pages will be rendered.

Possible approaches include:

- Client-Side Rendering (CSR)
- Server-Side Rendering (SSR)
- Hybrid Rendering

The choice affects performance, SEO, and user experience.

Architecture



Views Layer

- The Views layer contains all UI components that users directly interact with, such as the Feed, Post, Comment, and Image Viewer components. Its primary responsibility is to display data and collect user input.

- Views should focus only on presentation logic. They should not contain business logic or API communication code, which helps keep components simple and easier to maintain.
- Whenever a user performs an action (liking a post, submitting a comment, refreshing the feed), the view forwards that action to the appropriate controller.

Controller Layer

- Controllers act as intermediaries between UI components and business logic. They receive events from views and determine how those events should be processed.
- They perform tasks such as input validation, state updates, and invoking the correct service methods. This prevents UI components from becoming overloaded with logic.
- For example, when a user clicks the "Like" button, the Reaction Controller receives the event and delegates the actual operation to the Reaction Service.

Service Layer

- The Service layer contains reusable business logic and all communication with backend APIs. Instead of making API calls directly from components, requests are routed through services.
- Services handle data fetching, request formatting, error handling, retries, and response transformation before the data reaches the UI.
- Examples include Feed Service, Post Service, Comment Service, and Reaction Service. This separation improves maintainability and makes testing significantly easier.

Client-Side Caching

- Modern applications use tools such as React Query, SWR, or Apollo Client to cache frequently accessed data and reduce unnecessary network requests.

- Cached data allows the application to display content instantly while background requests fetch the latest updates from the server.
- This is particularly useful for news feeds because users frequently revisit content while scrolling, and constantly fetching the same data would increase latency and server load.

Service Workers

- Service Workers are background scripts that run independently from the main application and enable advanced browser features.
- They support offline access, push notifications, background synchronization, and intelligent caching of application assets.
- By serving cached content during network interruptions, Service Workers improve reliability and provide a better user experience.

IndexedDB

- IndexedDB is a browser-based database designed for storing large amounts of structured data on the client side.
- It can store feed data, user drafts, cached content, and other information that should remain available even after the browser is closed.
- Unlike Local Storage, IndexedDB supports larger datasets and is better suited for complex offline-first applications.

CDN Integration

- A Content Delivery Network (CDN) distributes static assets such as images, videos, JavaScript bundles, and stylesheets across geographically distributed servers.
- Users receive content from the nearest edge server instead of a centralized location, significantly reducing latency and improving load times.
- CDN usage is critical in social media systems because media content accounts for a large portion of overall traffic.

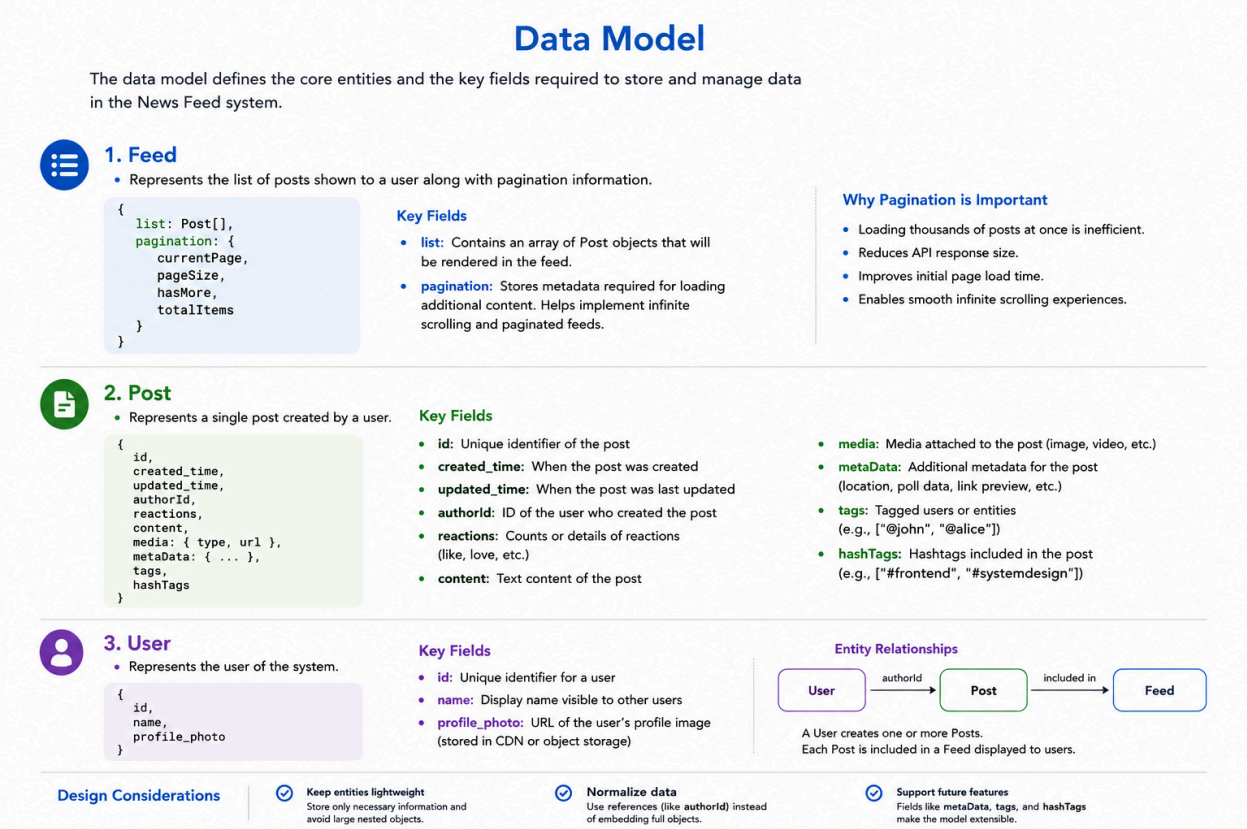
API Gateway

- The API Gateway serves as the single entry point between the frontend application and backend services.
- It handles authentication, request routing, rate limiting, monitoring, and other cross-cutting concerns before forwarding requests to the appropriate service.
- This approach simplifies frontend development because the client only communicates with one gateway rather than multiple backend services.

Backend Service Communication

- The frontend interacts with specialized backend services such as Auth Service, Feed Service, Post Service, and Comment Service through the API Gateway.
- Each service is responsible for a specific business capability, allowing teams to develop, deploy, and scale them independently.
- This microservice architecture improves flexibility and enables large-scale systems to handle millions of users efficiently.

Data Model



The Data Model defines how information is structured and exchanged between the frontend and backend. In a News Feed system, the primary entities are **Feed**, **Post**, and **User**. A well-designed data model improves performance, simplifies development, and makes the system easier to scale.

Feed Entity

The Feed entity represents the collection of posts displayed to a user. Instead of storing individual post details directly, it acts as a container that combines multiple posts with pagination metadata.

Structure

```
{
  list: Post[],
  pagination: {}
}
```

Key Components

- **list**
  - Contains an array of Post objects that will be rendered on the screen.
  - Each element represents a single post along with its associated information.
- **pagination**
  - Stores information required to load additional posts.
  - Supports features such as infinite scrolling and page-based navigation.

Why Feed Pagination Matters

- Fetching thousands of posts in a single request would increase response size and slow down rendering.
- Pagination allows data to be loaded gradually, reducing memory usage and improving user experience.
- Infinite scrolling becomes easier to implement because new posts can be requested whenever the user reaches the end of the current list.

Post Entity

The Post entity is the most important object in the News Feed system. Every piece of content displayed in the feed is represented as a Post object.

Structure

```
{
  id,
  created_time,
  updated_time,
  authorId,
  reactions,
  content,
  media: {
    type,
```

```
    url
  },
  metaData: {},
  tags,
  hashTags
}
```

Key Fields

id

- A unique identifier assigned to every post.
- Used when updating, deleting, sharing, or retrieving a specific post.
- Prevents ambiguity when multiple posts exist in the system.

created_time

- Stores the timestamp when the post was originally created.
- Helps sort posts chronologically in the feed.

updated_time

- Tracks when the post was last modified.
- Useful for showing edited content and synchronization.

authorId

- References the user who created the post.
- Establishes a relationship between the Post and User entities.

reactions

- Stores information about likes and other reactions.
- Can be represented either as counts or detailed reaction records.

Example:

```
{
  likes: 250,
  loves: 35,
  laughs: 12
}
```

content

- Contains the textual content of the post.
- May include plain text, mentions, hashtags, links, or formatted content.

media

- Stores media associated with the post.
- Usually contains the media type and the URL where the media is stored.

Example:

```
{
  type: "image",
  url: "https://cdn.example.com/photo.jpg"
}
```

Common media types include:

- Images
- Videos
- GIFs
- Documents

metaData

- Stores additional information that may not be required for every post.
- Provides flexibility for future features without changing the schema.

Examples include:

- Location information
- Poll data
- Shared post details
- Link previews

tags

- Represents users or entities tagged in the post.
- Enables mention functionality and notifications.

Example:

```
["@john", "@alice"]
```

hashTags

- Stores hashtags associated with the post.
- Helps with search, categorization, and trend analysis.

Example:

```
["#frontend", "#systemdesign"]
```

User Entity

The User entity represents a person using the platform. Every post, reaction, and comment is ultimately associated with a user.

Structure

```
{
  id,
  name,
  profile_photo
}
```

Key Fields

id

- Unique identifier for a user.
- Used throughout the system to establish relationships.

name

- Display name visible to other users.
- Appears in feeds, comments, profiles, and notifications.

profile_photo

- URL pointing to the user's profile image.
- Typically served through a CDN for faster delivery.

Example:

```
{
  profile_photo:
  "https://cdn.example.com/profile.jpg"
}
```

Entity Relationships

The three entities work together to create the complete News Feed experience.

```
User
  |
  | creates
  ▼
Post
  |
  | displayed in
  ▼
Feed
```

Relationship Flow

- A User creates one or more Posts.
- Each Post stores a reference to its creator using `authorId`.
- Multiple Posts are collected and organized into a Feed.
- The Feed is sent to the frontend and rendered for the user.

Design Considerations

Keep Entities Lightweight

- Only store information that is necessary for rendering and business operations.
- Avoid deeply nested objects whenever possible.
- Smaller objects reduce network transfer size and improve performance.

Normalize Data

- Store references instead of duplicating the same information repeatedly.
- For example, storing `authorId` is usually better than storing the complete user object inside every post.
- This reduces redundancy and simplifies updates.

Design for Future Growth

- Fields such as `metaData`, `tags`, and `hashTags` provide flexibility for future features.
- New capabilities can often be added without changing the entire schema.

Optimize for Feed Rendering

- News Feed systems are read-heavy applications.
- The data model should prioritize fast retrieval and efficient rendering of posts.
- Pagination and lightweight entities help maintain performance at scale.

Implementation Details

After defining the architecture and data model, the next step is implementing a News Feed system efficiently. Since feeds are highly interactive and can contain thousands of posts, frontend performance becomes a major concern. This section focuses on rendering strategies, pagination techniques, scrolling optimizations, media handling, and user experience improvements.

Rendering Approach

Rendering determines where HTML is generated before it is displayed to the user.

Server Side Rendering (SSR)

- In SSR, the server generates the HTML and sends it directly to the browser.
- The browser receives a fully rendered page, making content visible immediately.

Advantages

- Faster First Contentful Paint (FCP).
- Better SEO because search engines can directly read the HTML.
- Improved performance on slower devices.

Challenges

- Higher server load.
- Every request requires server-side rendering.

Common Use Cases

- Landing pages.
- Public profiles.
- Searchable content.

Client Side Rendering (CSR)

- The server initially sends a minimal HTML file and JavaScript bundle.
- JavaScript fetches data and renders the UI in the browser.

Advantages

- Reduced server workload.
- Rich and interactive user experiences.
- Easier state management after initial load.

Challenges

- Slower first load.
- Poor SEO without additional optimizations.

Common Use Cases

- Dashboards.
- Social media feeds.
- Authenticated applications.

Which Approach for News Feed?

- Most social media platforms use a hybrid approach.
- Initial page load may use SSR.
- Subsequent interactions are handled through CSR.
- This provides good SEO while maintaining responsiveness.

Infinite Scroll & Pagination

A News Feed can potentially contain millions of posts. Loading everything at once is impossible, so pagination is required.

Offset-Based Pagination

The client specifies:

```
{
  page: 2,
  size: 20
}
```

Working

```
Page 1 → Posts 1-20
Page 2 → Posts 21-40
Page 3 → Posts 41-60
```

Advantages

- Easy to implement.
- Works well for small datasets.

Problems

- Performance decreases with large datasets.
- New posts can shift results causing duplicates or missing records.

Example:

```
User loads Page 1

New post inserted

Page 2 now starts at a different position
```

This creates inconsistencies.

Cursor-Based Pagination

Instead of page numbers, the client sends a cursor.

```
{
  cursor: "post_12345",
```

```
  size: 20
}
```

Working

```
Fetch 20 posts
↓
Last Post ID = post_12345
↓
Use post_12345 as next cursor
↓
Fetch next 20 posts
```

Advantages

- More scalable.
- No duplicate records.
- Handles continuously changing feeds better.

Why Social Media Uses It

Platforms like Facebook, Twitter, Instagram, and LinkedIn frequently receive new posts every second. Cursor-based pagination ensures feed consistency despite constant updates.

Infinite Loading

Infinite loading automatically fetches more content as users scroll.

Scroll Event Approach

The application listens for scroll events.

```
window.addEventListener("scroll", ...)
```

When the user reaches near the bottom:

```
Scroll
↓
Detect Bottom
↓
Fetch More Posts
```

Problems

- Fires frequently.
- Can affect performance.
- Requires manual calculations.

Intersection Observer API

A modern browser API that observes when an element enters the viewport.

Working

```
Last Feed Item
       ↓
Becomes Visible
       ↓
Observer Triggered
       ↓
Fetch Next Page
```

Benefits

- More efficient than scroll listeners.
- Better performance.
- Less CPU usage.

Industry Practice

Most modern applications prefer Intersection Observer over traditional scroll events.

Virtualization

A feed may contain thousands of posts. Rendering all of them would overload the browser.

Virtualization solves this problem.

Concept

Instead of rendering every item:

```
10000 Posts
```

Only render:

```
Visible Posts
+
Small Buffer
```

Example:

```
User sees Posts 50-70

Render:
40-80

Ignore:
1-39
81-10000
```

Benefits

Less Browser Painting

- Fewer DOM elements.
- Less rendering work.
- Better scrolling performance.

Faster Reconciliation

- Virtual DOM compares fewer nodes.
- React updates become faster.

Lightweight Real DOM

- Smaller DOM tree.
- Reduced memory consumption.

Popular Libraries

- react-window
- react-virtualized
- TanStack Virtual

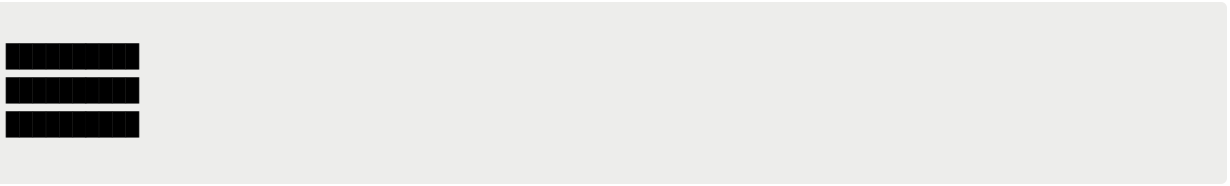
Shimmer Loading

Shimmer loading displays placeholders while content is loading.

Instead of showing:

```
Loading...
```

Show:



with animation.

Benefits

- Improves perceived performance.
- Users feel content is loading faster.

- Reduces layout shifts.

Common Implementations

SVG

- Highly customizable.
- Scalable graphics.

CSS3

- Most common implementation.
- Lightweight and easy to build.

GIF

- Simple but generally avoided.
- Larger file size.

Preserving Scroll Position

Users frequently navigate away from feeds and return later.

Without scroll preservation:

```
Scroll to Post #500
↓
Open Post
↓
Back
↓
Returned to Top
```

Poor user experience.

Solution

Store:

- Scroll offset.
- Cached feed data.

When user returns:

```
Restore Feed Data
↓
Restore Scroll Position
↓
Continue Browsing
```

Why SPAs Handle This Better

Single Page Applications can keep feed state in memory.

```
SPA
↓
Cached Data
↓
No Additional Server Request
```

Mentions & Hashtags

Modern feeds support:

```
@username
#frontend
```

Why Textarea Isn't Enough

Traditional textarea only supports plain text.

```
<textarea></textarea>
```

Cannot easily:

- Highlight mentions.

- Render hashtags.
- Support rich formatting.

content editable

A common approach:

```
<div contenteditable="true"></div>
```

Benefits

- Rich text editing.
- Inline mentions.
- Dynamic hashtag highlighting.

WYSIWYG Editors

"What You See Is What You Get" editors provide rich editing experiences.

Examples:

- Draft.js
- Slate.js
- Lexical
- Quill

Security Concern

User-generated HTML can introduce:

```
<script>alert("XSS")</script>
```

Always sanitize content before rendering.

Optimistic Updates

Users expect instant feedback.

Without optimistic updates:

```
Click Like
↓
Wait for Server
↓
UI Updates
```

Feels slow.

Optimistic Approach

```
Click Like
↓
Update UI Immediately
↓
Send API Request
↓
Success → Keep Change
Failure → Rollback
```

Benefits

- Better UX.
- Faster perceived performance.
- Common in social media applications.

Popular Libraries

- React Query
- SWR
- Apollo Client

These libraries provide built-in support for optimistic updates.

Timestamp Rendering

Posts usually display:

```
2 minutes ago
5 hours ago
Yesterday
```

instead of raw timestamps.

Server Side

Server stores timestamps in UTC.

Example:

```
2025-06-10T10:30:00Z
```

Client Side

Browser converts UTC into local time.

Common API

```
Intl.DateTimeFormat()
```

Benefits

- Supports multiple locales.
- Handles timezone differences automatically.

Relative Time Rendering

Instead of:

```
10 June 2025 10:30 AM
```

show:

```
2 minutes ago
3 hours ago
Yesterday
```

This is more meaningful for feed-based applications.

Icon Rendering

Icons are used throughout the feed for likes, comments, sharing, bookmarks, and notifications.

SVG Icons

Advantages

- Scalable without quality loss.
- Small file size.
- Easy styling.

Recommended Choice

Most modern applications use SVGs.

Separate Images

```
<img src="like.png" />
```

Problems

- Multiple network requests.
- Harder to customize.

Font Icons

Examples:

- Font Awesome
- Material Icons

Advantages

- Easy to use.

Limitations

- Less flexible than SVGs.
- Accessibility concerns.

Sprite Images

Multiple icons stored in one image.

Benefits

- Reduces HTTP requests.

Limitation

- More difficult to maintain than SVGs.

Image Rendering

Images form a significant portion of social media traffic.

Optimizing image delivery has a huge impact on performance.

CDN

Images should be served through a CDN.

Benefits

- Lower latency.
- Faster downloads.

- Reduced server load.

WebP Format

Modern image format providing:

- Smaller file sizes.
- Better compression.
- Faster loading.

Advantage

Typically 25–35% smaller than JPEG or PNG.

Responsive Images (src set)

Different devices require different image sizes.

Example:

```
<img
  src="small.jpg"
  srcset="small.jpg 400w,
          medium.jpg 800w,
          large.jpg 1200w"
/>
```

The browser automatically chooses the most appropriate image.

DPI Awareness

High-density screens require higher-resolution assets.

Examples:

- Retina Displays
- High-DPI Mobile Devices

Using device-specific images improves visual quality.

Progressive & Prefetch Loading

Instead of loading a full-resolution image immediately:

```
Blurred Preview
↓
Low Resolution
↓
High Resolution
```

Benefits

- Faster perceived loading.
- Better user experience.
- Reduced layout shifts.

Commenting System Considerations

Comments are often more complex than posts because they can contain nested replies.

Recursive Rendering

Comments form a tree structure.

```
Comment
 ├─ Reply
 │   ├─ Reply
 │   └─ Reply
 └─ Reply
```

Recursive components are commonly used to render nested discussions.

Cursor-Based Pagination

Large comment sections may contain thousands of replies.

Cursor-based pagination prevents excessive loading and improves performance.

Optimistic Comment Creation

New comments should appear instantly before the server confirms the operation.

This improves responsiveness and user engagement.

Lazy Loading Features

Additional features should load only when needed:

- Emoji pickers
- Mention suggestions
- Tag suggestions
- Rich formatting tools

This keeps the initial bundle size smaller and improves page load performance.